Minggu 3:

Enkapsulasi & Relasi antar Objek

Pemrograman Berorientasi Objek
Departemen Informatika
Universitas Diponegoro

Outline

1. Enkapsulasi
2. Access Modifier
3. Relasi Antar Objek

01
Enkapsulasi

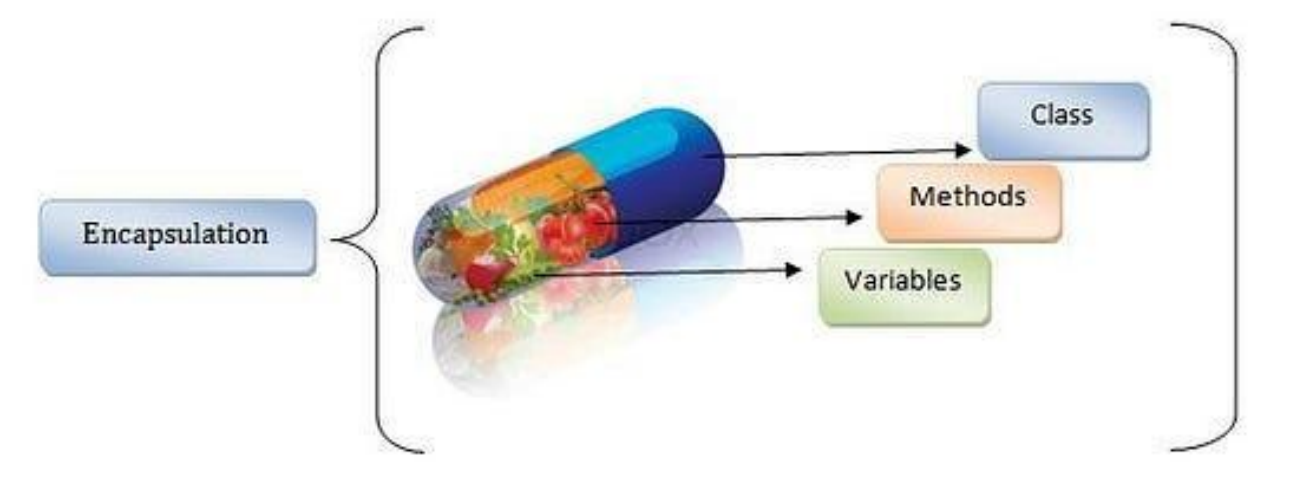
Pendahuluan

- Komunikasi melalui akses langsung data dimungkinkan, tetapi harus diminimkan.
  - Melalui akses public
  - Melalui akses static
- Lebih banyak akses statik, program yang dihasilkan semakin bersifat prosedural.
- Data semestinya diakses melalui operasi/method □ **enkapsulasi**.

Encapsulation

- Membungkus data dan method ke dalam single unit, yaitu class.
- Keuntungan enkapsulasi:
  - Memberikan batasan terhadap atribut-atribut objek agar tidak dapat diakses secara langsung oleh objek lainnya □ mendukung *information hiding*.
  - Mem-binding data dengan method yang bekerja pada data tersebut.
  - Modularitas: objek dapat dikelola secara independen.

Encapsulation



http://miro.medium.com/max/552/0*m8rZxUoWejR-ARJB.jpeg

Data/ Information Hiding

- Data hiding menyembunyikan atribut objek dari akses luar.
- Data hiding dapat diterapkan salah satunya dengan mengatur *visibility/access modifier* pada atribut dalam sebuah class.
- *Visibility modifiers* menunjukkan level akses terhadap atribut maupun method dari luar class.
- Jika *visibility modifier* pada class, atribut, atau method tidak ditentukan, maka yang berlaku adalah default.

Data/ Information Hiding

- Data hiding menyembunyikan atribut objek dari akses luar.
- Data hiding dapat diterapkan salah satunya dengan mengatur *visibility/access modifier* pada atribut dalam sebuah class.
- *Visibility modifiers* menunjukkan level akses terhadap atribut maupun method dari luar class.
- Jika *visibility modifier* pada class, atribut, atau method tidak ditentukan, maka yang berlaku adalah default.

Visibility Modifiers

Adornment	Visibility Name	Semantics
+	Public visibility	Any element that can access the class can access any of its features with public visibility
–	Private visibility	Only operations within the class can access features with private visibility
#	Protected visibility	Only operations within the class, or within children of the class, can access features with protected visibility
~	Package visibility	Any element that is in the same package as the class, or in a nested subpackage, can access any of its features with package visibility

Implementasi Information Hiding

- Menggunakan *visibility modifier* `private` untuk **atribut** class, sehingga nilai atribut hanya dapat diakses di dalam definisi class.
- Menggunakan *visibility modifier* `public` untuk **method** class, sehingga dapat diakses di luar definisi class.
- Membuat metode selector (*getter method*) untuk mengakses nilai atribut dari luar definisi class.
- Membuat metode mutator (*setter method*) untuk mengeset nilai atribut dari luar definisi class.
- **Constructor** umumnya diberikan *visibility modifier* `public`, kecuali jika ingin mencegah instansiasi objek secara bebas.

UML Class Diagram

The - sign indicates a private modifier

Circle	
-radius: double	The radius of this circle (default: 1.0).
-numberOfObjects: int	The number of circle objects created.
+Circle()	Constructs a default circle object.
+Circle(radius: double)	Constructs a circle object with the specified radius.
+getRadius(): double	Returns the radius of this circle.
+setRadius(radius: double): void	Sets a new radius for this circle.
+getNumberOfObjects(): int	Returns the number of circle objects created.
+getArea(): double	Returns the area of this circle.

FIGURE 9.17 The `Circle` class encapsulates circle properties and provides getter/setter and other methods.

Metode Selektor & Mutator

- Digunakan untuk mendapatkan (selektor) dan memanipulasi (mutator) satu atau beberapa atribut.
- Kesepakatan secara umum, selektor menggunakan `get` sedangkan mutator menggunakan kata `set`.
  - `public function getPropertyName(): returnType`
  - `public procedure setPropertyName(input propertyValue: dataType)`
- Untuk data bertipe boolean, get diganti dengan `is`.
  - `pubic function isPropertyName(): boolean`

Metode Selektor & Mutator (Java)

A getter method has the following signature:

```
public returnType getPropertyName()
```

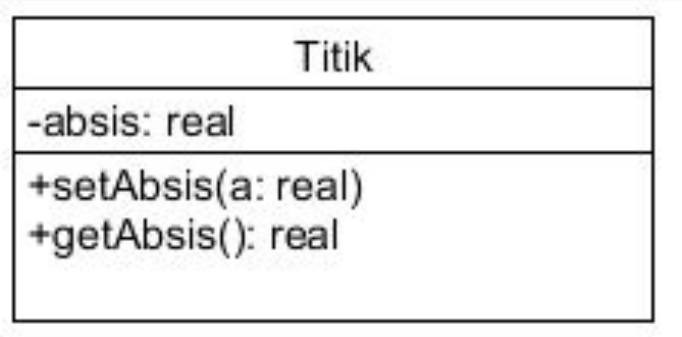
If the **returnType** is **boolean**, the getter method should be defined as follows by convention:

```
public boolean isPropertyName()
```

A setter method has the following signature:

```
public void setPropertyName(dataType propertyValue)
```

Metode Selektor & Mutator




```
public class Titik {
    private absis: real

    public procedure setAbsis(input a: real){
        absis □ a;
    }
    public getAbsis(): real{
        □ absis;
    }
}
```

Contoh Enkapsulasi pada Class Titik

```
10  public class Titik {
11      //atribut
12      private double absis;
13      private double ordinat;
14      //static int numberOfPoints;
15
16      //konstruktor
17      //membuat objek titik dengan inisialisasi nilai absis dan ordinat
18      public Titik(double absis, double ordinat){
19          this.absis = absis;
20          this.ordinat = ordinat;
21      }
22
23      //membuat objek titik dengan inisialisasi absis 0 dan ordinat 0
24      public Titik(){
25          this(0,0);
26      }
```

```java
    //method
    //fungsi selektor untuk mendapatkan nilai atribut absis
    public double getOrdinat(){
        return this.ordinat;
    }

    //fungsi selektor untuk mendapatkan nilai atribut ordinat
    public double getAbsis(){
        return this.absis;
    }

    //prosedur untuk mengeset nilai atribut absis dengan nilai yang baru
    public void setAbsis(double absis){
        this.absis = absis;
    }

    //prosedur untuk mengeset nilai atribut ordinat dengan nilai yang baru
    public void setOrdinat(double ordinat){
        this.ordinat = ordinat;
    }
}
```

- Kode manakah yang akan bermasalah saat dijalankan?

```
Titik t1 = new Titik()
System.out.println(t1.absis)
System.out.println(t1.ordinat)
System.out.println(t1.getAbsis())
System.out.println(t1.getOrdinat())
```

02

Access Modifier

Dalam bahasa Java

Modifier (in Java)

- Beberapa jenis modifier dalam Bahasa Java:

Nama Modifier	Berlaku untuk	Jenis
Access modifier	Class, method, variable	Public, protected, private, default
Final modifier	Class, method, variable	Final
Static modifier	Variable dan method	Static
Abstract modifier	Class dan method	Abstract

Access Modifier (in Java)

Modifier	Class dan Interface	Method dan Variable
Default	Tampak di paketnya	Dapat diakses oleh method di class lain yang sepaket. Diwarisi oleh subclass-nya di paket yang sama dengan class-nya.
Public	Tampak dimanapun	Dapat diakses dimanapun. Diwarisi oleh semua subclass-nya.
Protected	Tidak dapat diterapkan	Dapat diakses oleh method di class lain yang sepaket. Diwarisi oleh semua subclass-nya.
Private	Tidak dapat diterapkan	Tidak dapat diakses oleh class lain. Tidak diwarisi oleh subclass-nya.

Abstract Modifier (in Java)

Berlaku untuk	Keterangan
Class	Class tidak dapat diinstantiasi. Class tidak mempunyai contructor.
Interface	Opsional untuk diberikan pada interface karena interface secara inheren adalah abstrak.
Method	Tidak ada badan method yang didefinisikan. Method memerlukan class konkret yang merupakan subclass yang akan mengimplementasikan method abstract.
Variable	Tidak dapat diterapkan.

Final Modifier (in Java)

Berlaku untuk	Keterangan
Class	Class menjadi tidak dapat digunakan untuk menurunkan class yang baru.
Interface	Tidak dapat diterapkan.
Method	Method tidak dapat ditimpa oleh method di subclass-nya.
Variable	Berperilaku sebagai konstanta (tidak dapat diubah nilanya).

Static Modifier (in Java)

Berlaku untuk	Keterangan
Class	Tidak dapat diterapkan.
Interface	Tidak dapat diterapkan.
Method	Mendefinisikan method milik class, sehingga tidak memerlukan instance object untuk menjalankannya. Method ini tidak dapat menjalankan method yang tidak static serta tidak dapat mengacu variable yang bukan static.
Variable	Mendefinisikan variable milik class, sehingga tidak memerlukan instance object untuk mengacunya. Variabel ini dapat digunakan Bersama oleh semua instance object.

Static Modifier (in Java)

Berlaku untuk	Keterangan
Class	Tidak dapat diterapkan.
Interface	Tidak dapat diterapkan.
Method	Mendefinisikan method milik class, sehingga tidak memerlukan instance object untuk menjalankannya. Method ini tidak dapat menjalankan method yang tidak static serta tidak dapat mengacu variable yang bukan static.
Variable	Mendefinisikan variable milik class, sehingga tidak memerlukan instance object untuk mengacunya. Variabel ini dapat digunakan Bersama oleh semua instance object.

Latihan 1

- Sebutkan atribut dan method yang dapat diakses pada class C2 dan C3!

```java
package p1;

public class C1 {
  public int x;
  int y;
  private int z;

  public void m1() {
  }
  void m2() {
  }
  private void m3() {
  }
}
```

```java
package p1;

public class C2 {
  void aMethod() {
    C1 o = new C1();
    can access o.x;
    can access o.y;
    cannot access o.z;

    can invoke o.m1();
    can invoke o.m2();
    cannot invoke o.m3();
  }
}
```

```java
package p2;

public class C3 {
  void aMethod() {
    C1 o = new C1();
    can access o.x;
    cannot access o.y;
    cannot access o.z;

    can invoke o.m1();
    cannot invoke o.m2();
    cannot invoke o.m3();
  }
}
```

Latihan 1

- Sebutkan atribut dan method yang dapat diakses pada class C2 dan C3!

Latihan 2

- Apakah objek C1 dapat dibuat di class C2?
- Apakah objek C1 dapat dibuat di class C3?

```
package p1;

class C1 {
  ...
}
```

```
package p1;

public class C2 {
  can access C1
}
```

```
package p2;

public class C3 {
  cannot access C1;
  can access C2;
}
```

Latihan 3

- Bagaimana hasil pemanggilan atribut dan method di class Test seperti yang ditunjuk pada gambar?

```java
public class C {
  private boolean x;

  public static void main(String[] args) {
    C c = new C();
    System.out.println(c.x);
    System.out.println(c.convert());
  }

  private int convert() {
    return x ? 1 : -1;
  }
}
```

```java
public class Test {
  public static void main(String[] args) {
    C c = new C();
    System.out.println(c.x);
    System.out.println(c.convert());
  }
}
```

03

Relasi Antar Objek

Link

- Dalam OOP, objek perlu berkomunikasi dengan objek lainnya.
- Link merupakan semantic connection antara 2 objek yang memungkinkan message dapat dikirim dari satu objek ke objek lainnya.

Hubungan antar Class

- Hubungan antar class (Inggriani Liem, Diktat Kuliah PBO)
  - Hubungan Client-Supplier
  - Hubungan Inheritance
- Relasi antar class (UML Class Diagram)
  - Association
  - Aggregation
  - Composition
  - Inheritance
  - Dependency

Hubungan Client-Supplier

- Supplier menyediakan sejumlah services yang dapat dipakai oleh Client.
- Client harus memenuhi “kontrak” berupa aturan/prekondisi yang tertulis sebelum memakai services yang disediakan oleh Supplier.
- Kelas A adalah Client dari kelas B dan B adalah Supplier dari Kelas A jika A mengandung definisi entitas e: B.
- Entitas dapat berupa :
  - atribut
  - argumen formal pada fungsi/prosedur
  - hasil evaluasi fungsi (*return value*)

Hubungan Inheritance

- Kelas turunan (descendant/ child) mewarisi kelas leluhur (parent) sehingga “semua” atribut dan method kelas parent akan “dibawa” secara intrinsik menjadi bagian dari kelas turunan.
- Di samping itu, biasanya pemroses bahasa menyediakan sarana untuk:
  - Menambah feature baru
  - Mengubah atau menggantikan feature yang diwarisi
  - Menghapus feature yang diwarisi (berpotensi menimbulkan konsekuensi yang berbahaya)
  - Menentukan feauture yang masih deferred (belum terdefinisi)

Hubungan antar Class

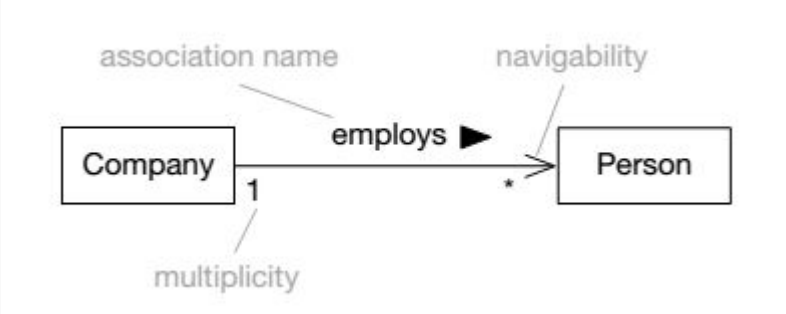
- Hubungan antar class (Inggriani Liem, Diktat Kuliah PBO)
  - Hubungan Client-Supplier
  - Hubungan Inheritance
- Relasi antar class (UML Class Diagram)
  - Association
  - Aggregation
  - Composition
  - Inheritance
  - Dependency

Association

- Asosiasi menunjukkan relasi antara satu class dengan class lainnya.
  - Asosiasi □ relasi antar class
  - Link □ koneksi antar objek
- Sebuah asosiasi dapat mempunyai:
  - association name (sebaiknya dalam kata kerja)
  - role names
  - multiplicity
  - navigability

Association

- **Association name**
- Contoh:
  - "A Company employs many Persons."
  - "Each Person is employed by exactly one Company (at any point in time)."



Association

- **Role names**
- Contoh:
  - "A Company employs many Persons."
  - "A Person is employed by exactly one Company (at any point in time)."

Association

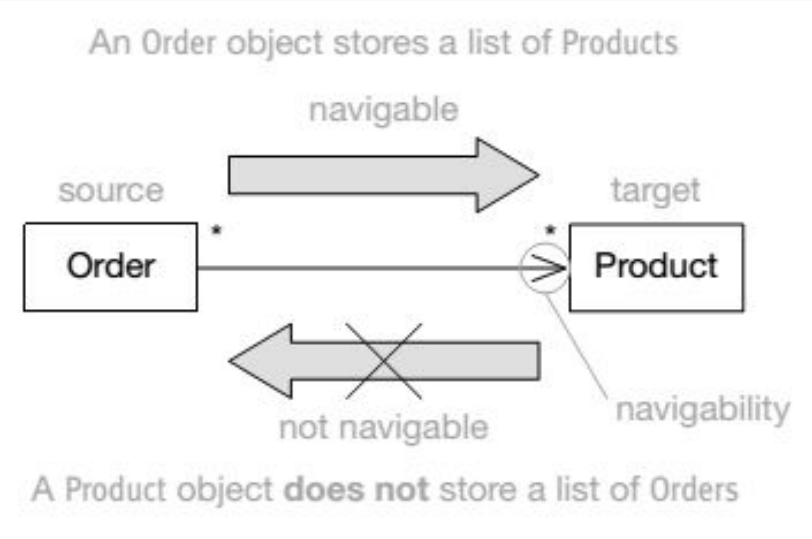
- **Multiplicity** □ memberikan constraint pada jumlah objek dalam sebuah class yang terlibat dalam relasi dengan objek lain pada waktu tertentu.

Table 9.1

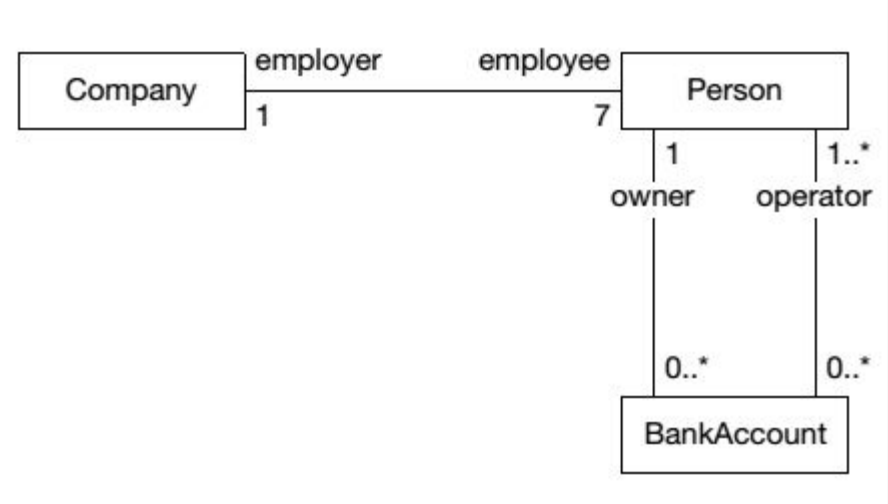
Adornment	Semantics
0..1	Zero or 1
1	Exactly 1
0..*	Zero or more
*	Zero or more
1..*	1 or more
1..6	1 to 6
1..3,7..10,15, 19..*	1 to 3 *or* 7 to 10 *or* 15 exactly *or* 19 to many

Association

- **Navigability** □ menunjukkan arah komunikasi (pengiriman pesan) dari *source object* ke *target object*.
- "Order objects can send messages to Product objects, but not vice versa."



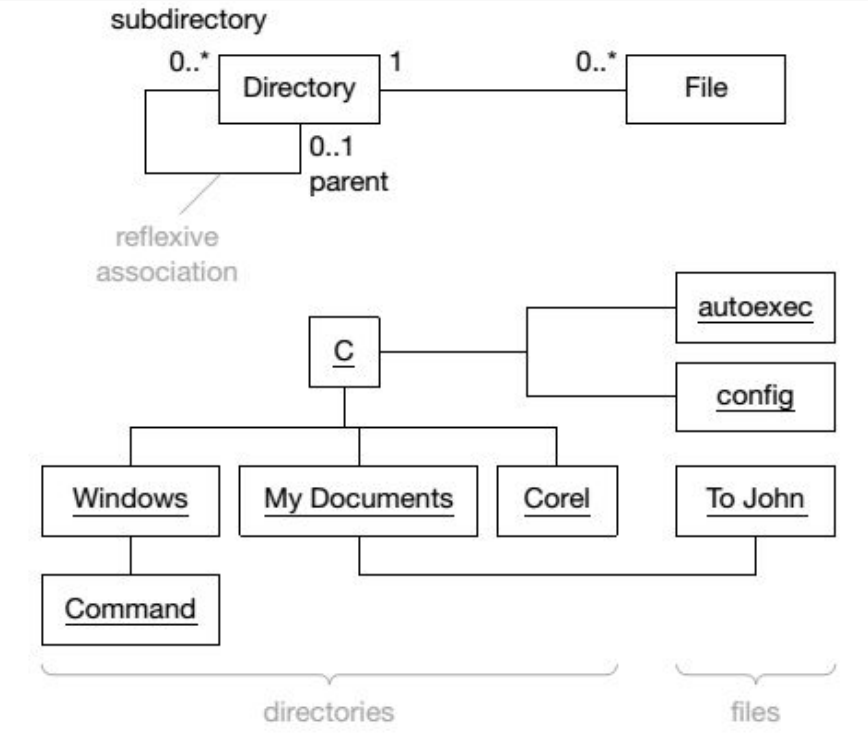
Association



- a BankAccount can have exactly one owner;
- a BankAccount can have one or many operators;
- a Person may have zero to many BankAccounts;
- a Person may operate zero to many BankAccounts.

Reflexive Association

- Objek dari sebuah class memiliki asosiasi dengan objek lain dari class yang sama.
- "Each Directory object can have links to zero or more Directory objects that play the role subdirectory, and to zero or one Directory object that plays the role parent."



Association & Class Attributes

```
public class House
{
        private Address address;
}
```

If a navigable relationship has a role name, then it is as though the source class has a pseudo-attribute with the same name as the role name and the same type as the target class

- Objek dari source class memiliki pseudo-attribute dari target class.
- Jika multiplicity lebih dari 1 dapat menggunakan salah satu cara berikut:
  - Array of object)
  - Object collection

Association & Class Attributes

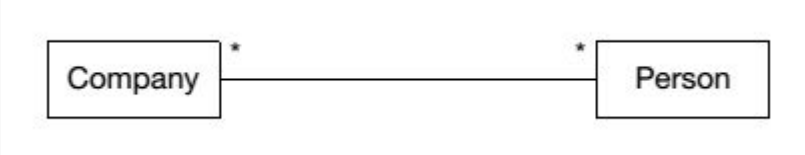
- Asosiasi digunakan hanya ketika kelas target merupakan bagian penting dari business model.
- Jika tidak, maka cukup memodelkan hubungan menggunakan atribut.
- Class penting adalah kelas bisnis yang menggambarkan bagian dari business domain.
- Class yang tidak penting adalah komponen library seperti class String, Date, dan Time.

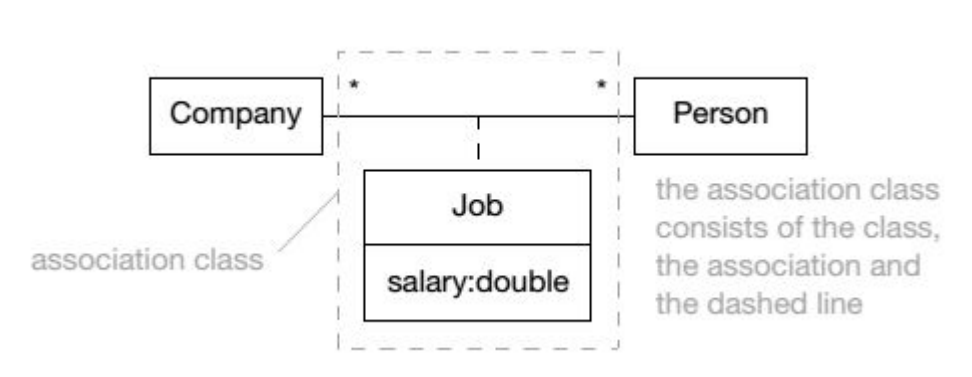
Association & Class Attributes

- Asosiasi digunakan hanya ketika kelas target merupakan bagian penting dari business model.
- Jika tidak, maka cukup memodelkan hubungan menggunakan atribut.
- Class penting adalah kelas bisnis yang menggambarkan bagian dari business domain.
- Class yang tidak penting adalah komponen library seperti class String, Date, dan Time.

Association Class

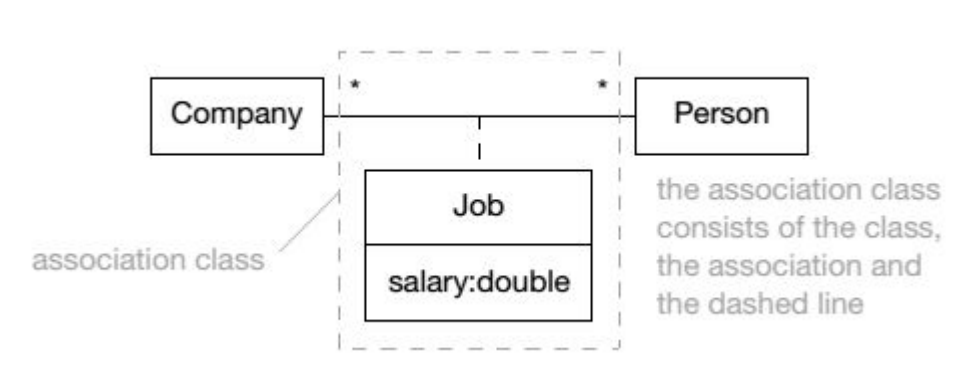
- Bagiamana apabila setiap Person dapat bekerja di banyak Company tapi mendapatkan gaji sendiri pada masing-masing Company?

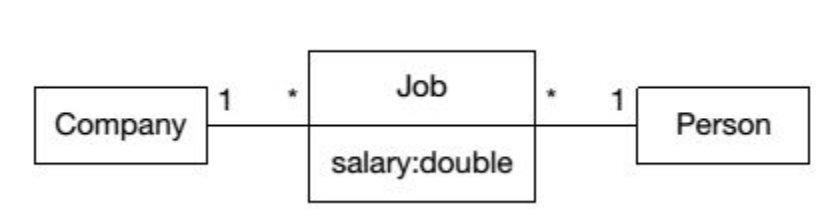




Association Class

- Alternatif: menggunakan class Job.





Association

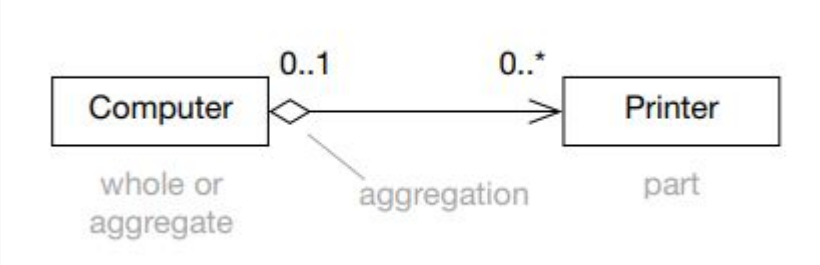
- Asosiasi biasanya adalah relasi yang diidentifikasi pada tahap analisis.
- Pada tahap desain, asosiasi harus diperjelas ke dalam bentuk agregasi atau komposisi.

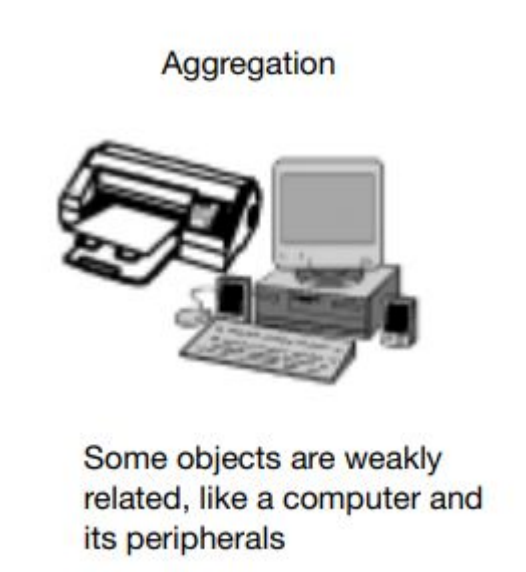
Agregation

- Agregasi adalah jenis hubungan keseluruhan-bagian (*whole-part*) dimana agregat terdiri dari banyak bagian.
- *Whole-object* menggunakan *service* objek lain (*part-object*).
- *Whole-object* bersifat dominan dan mengendalikan hubungan,
- *Part-object* lebih pasif, hanya melayani permintaan dari *whole-object*.

Agregation

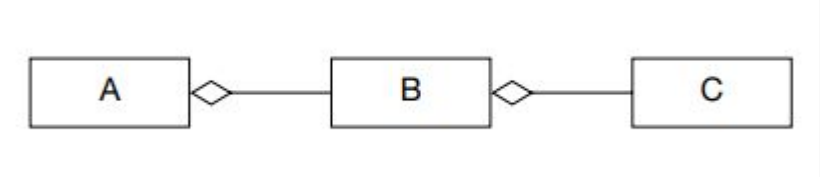
- a Computer may be attached to 0 or more Printers;
- at any one point in time a Printer is connected to 0 or 1 Computer;
- over time, many Computers may use a given Printer;
- the Printer may exist even if there are no attached Computers;
- the Printer is independent of the Computer.





Agregation

- Agregat terkadang bisa eksis secara independen dari bagian-bagiannya, terkadang tidak.
- Parts dapat eksis secara independen dari agregatnya.
- Agregat dalam beberapa hal tidak lengkap jika beberapa bagiannya hilang.
- Aggregation is transitive.
- Aggregation is asymmetric.

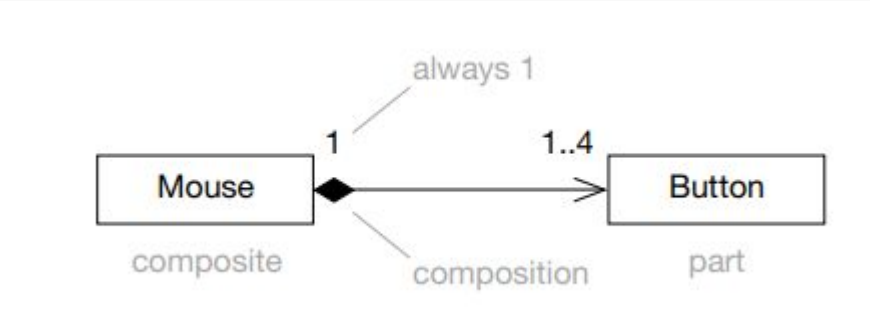


Composition

- Bagian-bagian tersebut termasuk dalam satu komposit pada satu waktu;
- Komposit mempunyai tanggung jawab penuh atas disposisi seluruh bagiannya (tanggung jawab atas pembuatan dan penghancurannya).
- Jika komposit tersebut hancur, maka komposit tersebut harus menghancurkan seluruh bagiannya, atau menyerahkan tanggung jawab atas komponen tersebut kepada objek lain.

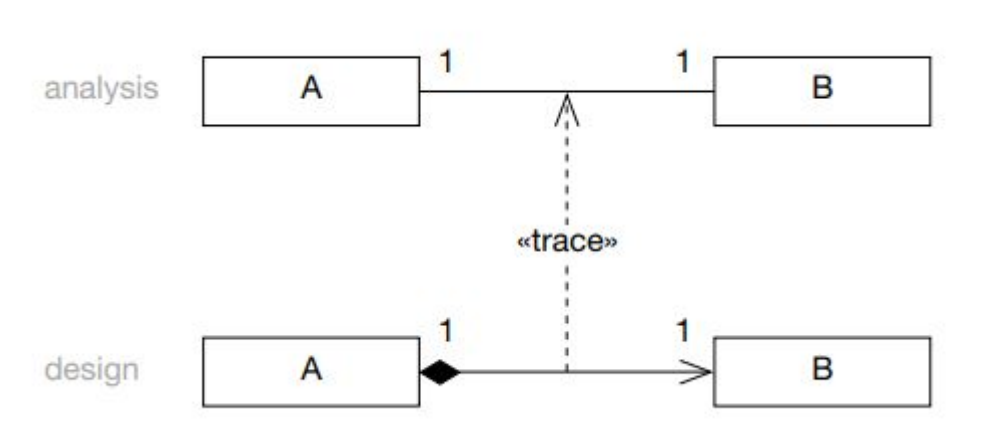
Composition

- Button objects have no independent existence apart from their owning Mouse object.
- If you destroy the Mouse object, you destroy its Button objects as they are an integral part of it.
- Each Buttonobject can belong to exactly one Mouseobject.



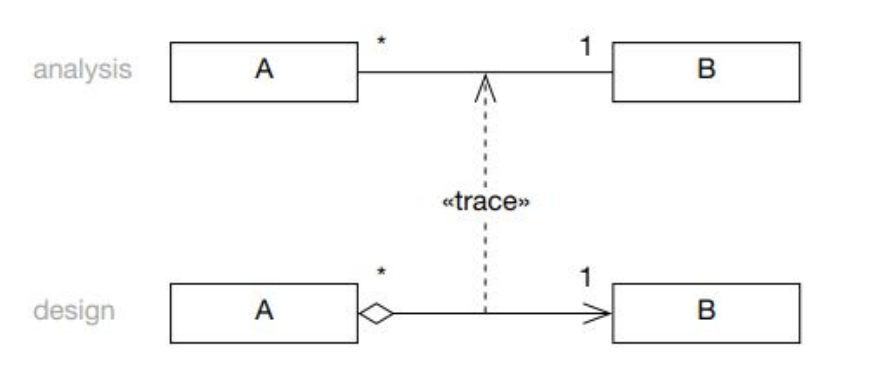
Multiplicity

- One-to-one association
  - One-to-one association hampir selalu diterjemahkan ke dalam komposisi.
  - B dapat dianggap sebagai atribut jika B bukan merupakan class penting.



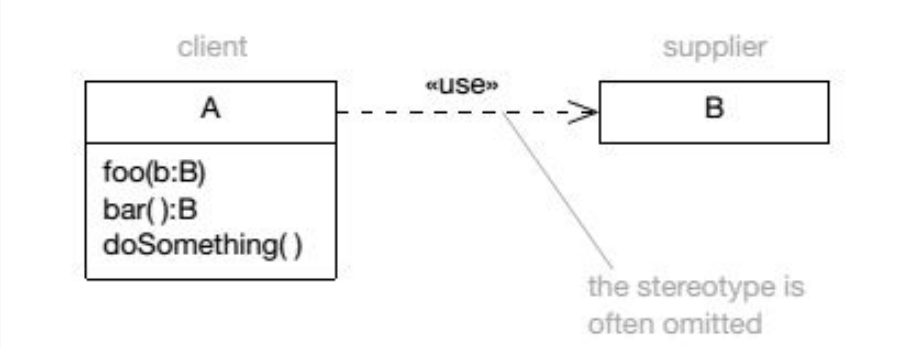
Multiplicity

- Many-to-one association
  - Jika part dapat digunakan bersama oleh whole lainnya, maka harus dijadikan agregasi.
  - Melibatkan Kumpulan objek □ dapat menggunakan array object atau object collection.



Dependency

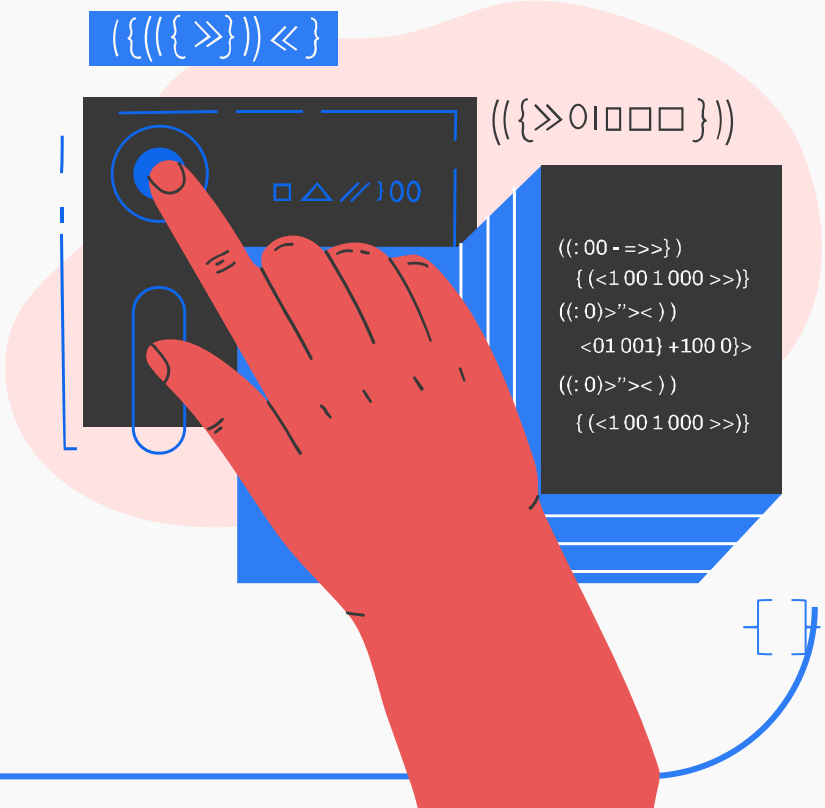
- Dependency merupakan relasi antara dua kelas jika perubahan definisi salah satu kelas (supplier) dapat menyebabkan perubahan pada kelas lainnya (client), tetapi tidak sebaliknya.
- Relasi dependency direpresentasikan dengan garis putus-putus dengan panah terbuka



Referensi

- Panji Wisnu Wirawan, Indra Waspada, Satriyo Adhy. 2018. ***Buku Ajar Pemrograman Berorientasi Objek***.
- Daniel Y. Liang, ***Introduction into Java Programming Comprehensive Version 10th Edition***, 2015, Pearson.
- Vaskaran Sarcar. 2020. ***Interactive Object-Oriented Programming in Java***.
- Jim Arlow & Ila Neustadt, UML and The Unified Process – Practical Object Oriented and Design, 2005, Addision Wesley.
- https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-class-diagram/

Thanks!



CREDITS: This presentation template was created by **Slidesgo**, and includes icons by **Flaticon**, and infographics & images by **Freepik**

Please keep this slide for attribution